Session 7 · Deck 7.B4

What More Parameters Buy; Order, and Its Absence

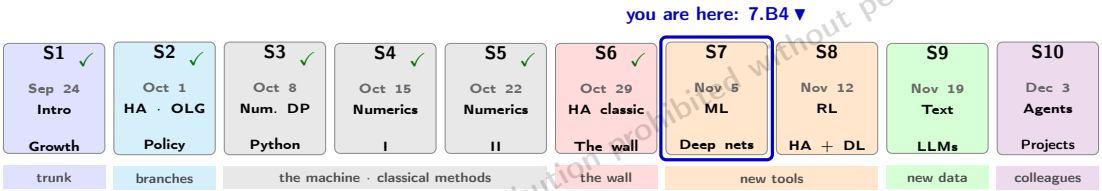
Zhigang Feng

Thursday, November 5, 2026

© Zhigang Feng Redistribution prohibited without permission

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are



Build it on paper → solve it on the machine → push it to the frontier with AI

Block A — learning a function: 7.A1 the learning problem and the neuron · 7.A2 activations and approximation · 7.A3 backpropagation and training. **Block B — using it:** 7.B1 PyTorch and a forecast · 7.B2 the growth model · 7.B3 architecture and shape · **7.B4** capacity and order.

© Zineb Fort
Redistribution prohibited without permission

Roadmap: Two Parts and an Appendix

The opposite move: no constraint at all, and more parameters than observations. Then order, and its absence.

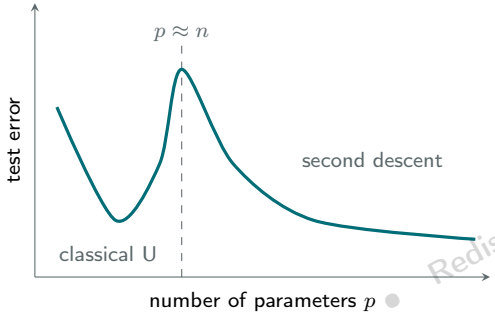
Part	Topic	The question it answers
1	What more parameters buy	With no constraint and more parameters than observations, which fit does training pick, and is that overfitting?
2	Order, and its absence	What do a sequence, and a set of agents where names do not matter, each claim about the input?
App.	One input, four layers	Can every number be checked by hand, and what does each layer refuse to notice?

© Zhiheng Feng • Redistribution prohibited without permission

Idea

7.B3 and this deck ask one question: **what decides the answer when the data do not?** In 7.B3 it is a restriction from theory; in Part 1 it is the optimizer; in Part 2 it is again a constraint on W, now about order.

More parameters than observations is not a defect. It is a claim to test out of sample.



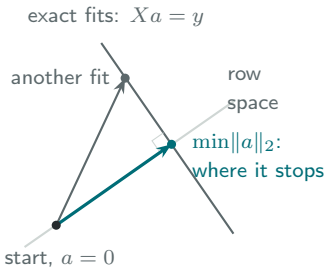
- Test error follows the textbook U, then peaks at the **interpolation threshold** $p \approx n$, where the model first fits the sample exactly.
- With $p \gg n$ it falls again: **double descent** (a schematic).
- In forecasting, $p > n$ can lower out-of-sample risk.
- **More parameters than observations is not a defect. It is a claim to test out of sample.**

© Zhigang Feng
Redistribution prohibited without permission

Measured on this deck's own example four frames on. M. Belkin, D. Hsu, S. Ma, S. Mandal, *PNAS* 116(32), 2019; Y. Liao, X. Ma, A. Neuhierl, Z. Shi, "Economic forecasts using many noises", arXiv 2312.05593, 2023.

1 · Why Gradient Descent Picks That Fit

The optimiser cannot move in directions the data never mention.



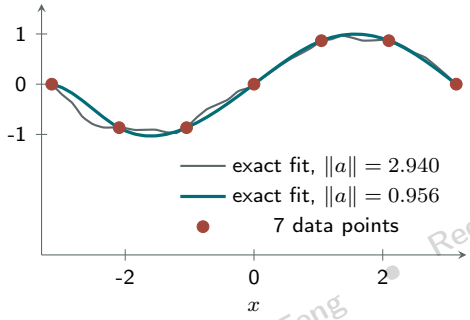
- Linear in the parameters: every gradient $\nabla_a L = X^\top(Xa - y)$, and so every step, lies in the **row space** of X.
- From $a = 0$ the iterate never leaves it, and one exact fit alone lies there: the smallest $\|a\|_2$. **The optimiser cannot move in directions the data never mention.**
- $\|\cdot\|_2$ is the selector, Euclidean because the steps are — not a smoothness claim. Multiplicative updates select ℓ_1 instead: the algorithm is part of the estimator.
- **With $p > n$ the algorithm picks the fit. Whether it generalises is empirical, not a theorem.**

© Zhigang Feng
Redistribution prohibited without permission

The min-norm fact above is elementary: the derivation is its proof. S. Gunasekar, J. Lee, D. Soudry, N. Srebro, "Characterizing implicit bias in terms of optimization geometry", ICML 2018.

Implicit Regularization, Stated Precisely

The optimiser picks among exact fits. No penalty was written.



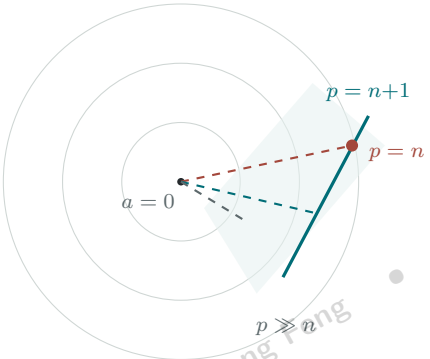
- Seven points of $\sin x$, 41 fixed features: the coefficients $a \in \mathbb{R}^{41}$ are not pinned down, and infinitely many fits are exact.
- Gradient descent from zero converges to the fit of **smallest Euclidean norm** (teal). No penalty was written.
- **The optimiser picks among exact fits. For deep networks: intuition, not a theorem.**

© Zhigang Feng Redistribution prohibited without permission

Lab 7, block B3. Soudry et al., *JMLR* 19, 2018 (the classification analogue: the iterate's *direction* converges to the maximum margin); Hastie et al., *Ann. Statist.* 50(2), 2022 (risk at $p > n$).

Why the Norm Moves: It Is a Minimum over a Set

More parameters enlarge the set of exact fits; more data shrink it.



Dashed: the smallest member of each set.

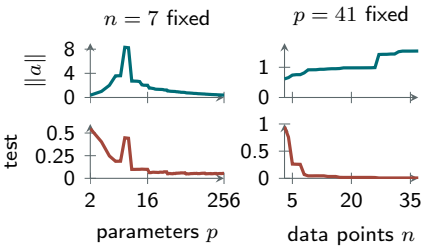
- $\|a\|$ is the *smallest* coefficient vector that fits. It is a minimum over a set, so whatever changes the set changes it.
- **More parameters** give more ways to fit: the set grows, so its minimum falls. **More data** give more to satisfy: the set shrinks, so its minimum rises.
- At $p = n$ the set is a *single point* — nothing to choose — and that is the peak. Below $p = n$ nothing fits exactly at all.

© Zhirang Peng
Redistribution prohibited without permission

Both directions follow from one argument; both are checked in Lab 7, block B3.

Two Levers, Measured

The norm falls with the lever that need not help, and rises with the one that always does.



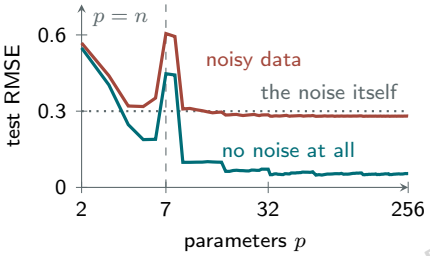
- **More** p (left): below $p = n$ nothing fits exactly; at $p = n$ the single fit costs 8.3; above it the norm falls to 0.4.
- **More** n (right): $0.62 \to 1.54$, and every fit is still exact.
- Test error follows neither: past $p = n$ it drops — the optimiser may now *choose* — then stalls near $p = 65$. More data never stops helping, $0.95 \to 0.006$.
- **The norm falls with the lever that need not help, and rises with the one that always does.**

© Zhigang Feng
Redistribution prohibited without permission

Lab 7, block B3. Nested hinges and nested data points, so each sweep adds to what came before.

Is That Just Overfitting?

No: every candidate ties at zero training error; only the tie-break differs.



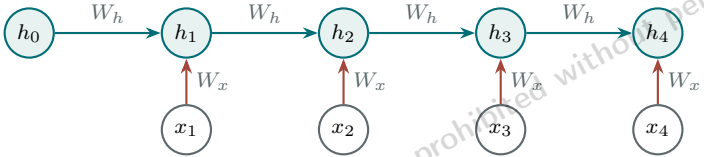
- No: the training error is **zero for every $p \geq n$ on both curves**, so a loss below the noise floor (7.A3) cannot tell them apart.
- **Overfitting** drags the model onto noise. Here every candidate ties at zero; only the tie-break differs.
- With $y = \sin x$ exactly there is nothing to overfit, yet $p = n$ gives 0.45 against 0.19 at $p = 6$. With noise the two stack and the peak rises to 0.61.
- **Overfitting is cured with less capacity; this, with a better tie-break or more data.**

© Zhigang Feng
Redistribution prohibited without permission

Lab 7, block B3; $n = 7$**, noise s.d.** 0.30**, mean over 200 draws.**

Recurrent Networks: the Hidden State Is a State Variable

The constraint: one law of motion at every date.



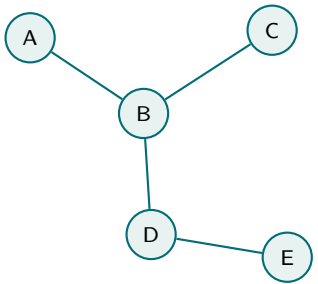
Redistribution prohibited without permission

- A **recurrent network** reads a sequence one date at a time: $h_t = \sigma(W_h h_{t-1} + W_x x_t + b)$, the same W_h, W_x at every date.
- The **hidden state** h_t summarises the history: a state-space model with an estimated transition.
- Going back T dates multiplies by W_h^T, which vanishes or explodes. An LSTM's gates make persistence depend on the state; an AR(1) has a constant ρ.
- **The constraint: one law of motion at every date.**

© Zhaonan Feng

S. Hochreiter and J. Schmidhuber, "Long short-term memory", *Neural Computation* 9(8), 1997; Miao & Feng, §9.6.

Renumbering the nodes cannot change the answer — and heterogeneous-agent models impose the same.



five banks; an edge is lending

- A **graph network** updates each node from itself and its neighbours' average, with one pair of matrices for all nodes: $h'_v = \sigma\big(W_{\mathrm{s}} h_v + W_{\mathrm{n}} \overline{h}_{N(v)} + b\big)$.
- Neighbours have no order, so the aggregate is a sum or a mean: **permutation invariance**. Renumbering nodes cannot change the answer.
- **Heterogeneous-agent models impose the same: prices depend on the distribution, not on who is who (6.A2's λ; Session 8 feeds it to a network).**

© Zhigang Feng · Redistribution prohibited without permission

T. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks", ICLR 2017; Miao & Feng, §9.6.

Takeaways

- **Part 1.** More parameters than observations is not overfitting: every candidate ties at zero training error, and only the tie-break differs. From zero, gradient descent picks the smallest exact fit; the norm falls with p and rises with n, and the test error follows neither.
- **Part 2.** A recurrent network's hidden state is a state variable with one law of motion at every date. A graph network's permutation invariance is the restriction heterogeneous-agent models impose: prices depend on the distribution, not on who is who.
- **Both halves, 7.B3's restrictions and this deck's unrestricted fits, are the same question: what decides the answer when the data do not?**

© Zhigang Feng · Reproduction prohibited without permission

Bridge: To Session 8

Session 7 learned a function from examples and then from an optimality condition, with the economics written into the architecture where theory supplies it. It solved the growth model by the Euler road.

Session 8 takes the other road — the Bellman equation, without a model of the environment — and then puts a whole distribution of households into the network's input, where permutation invariance stops being an analogy.

Next: Session 8

Reinforcement learning as dynamic programming without the model; deep value iteration and actor–critic; then heterogeneous agents at the frontier: the distribution as a network input, deep equilibrium nets, and DeepHAM.

Homework 2, C2 trains 7.B2's method on your HW1 model or on the Aiyagari household, and enforces the borrowing limit by penalty or by construction (7.B2, 7.B3).

© Zhigang Feng • Redistribution prohibited without permission

Appendix: One Input, Read Four Ways

The input never changes. The architecture is the claim about what its four numbers are.



Redistribution prohibited without permission

- Every layer below is the same $\sigma(Wx + b)$. They differ only in which entries of W are free, which are fixed at zero, and which are forced to be equal.
- Weights are chosen here, not trained, so every number on the next two frames can be checked by hand.
- **The input never changes. The architecture is the claim about what its four numbers are.**

© Zhigang Feng

Appendix: Four Layers, Four Answers

Three free numbers do the work of five, and only because the tying is true of the input.

	Rule	Output	Free
dense	$w = (\frac{1}{2}, -\frac{1}{2}, \frac{1}{2}, -\frac{1}{2})$	-1	5
convolution	filter $(1, -1)$, slid	$(-1, -1, -1)$	3
recurrent	$h_t = \frac{1}{2} h_{t-1} + x_t$	$(1,\ 2.5,\ 4.25,\ 6.125)$	3
graph	$h'_v = x_v + \frac{1}{2} \overline{x}_{N(v)}$	$(2,\ 3,\ 4.5,\ 5.5)$	3

1	-1	0	0
0	1	-1	0
0	0	1	-1

the convolution's W: 3×4, three numbers

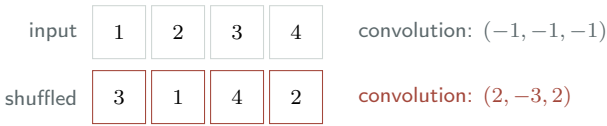
- Dense returns one number, the convolution three, the recurrent and graph layers four: the constraint changes the *shape* of the answer, not only the count of parameters.
- The convolution's W is written out on the right. Nothing was added to the dense layer — entries were set to zero and the survivors tied together.
- **Three free numbers do the work of five, and only because the tying is true of the input.**

© Zhigang Feng · Redistribution prohibited without permission

Every value asserted against the arithmetic in Lab 7, appendix (one input, four architectures).

Appendix: What Each One Refuses to Notice

A constraint is not a saving. It is a statement about the input that can be wrong.



the dense layer, with its weights re-labelled the same way, is unchanged

© Zhigang Feng · Redistribution prohibited without permission

- **Shuffle the input.** The convolution's answer is destroyed: it assumed positions 1 and 2 were neighbours, and now they are not. The dense layer never made that claim, so it learns the shuffled problem just as well — 7.B3's digit frame, as arithmetic.
- **Relabel the graph's nodes** $A\,B\,C\,D$ as $D\,C\,B\,A$: the output becomes $(5.5,\ 4.5,\ 3,\ 2)$, the same four numbers in the new order, and a sum readout stays 15 either way. That is permutation invariance, as a number.
- The recurrent layer weights the past geometrically, so any reordering of the dates changes the answer: order is the whole of its claim.
- **A constraint is not a saving. It is a statement about the input that can be wrong.**